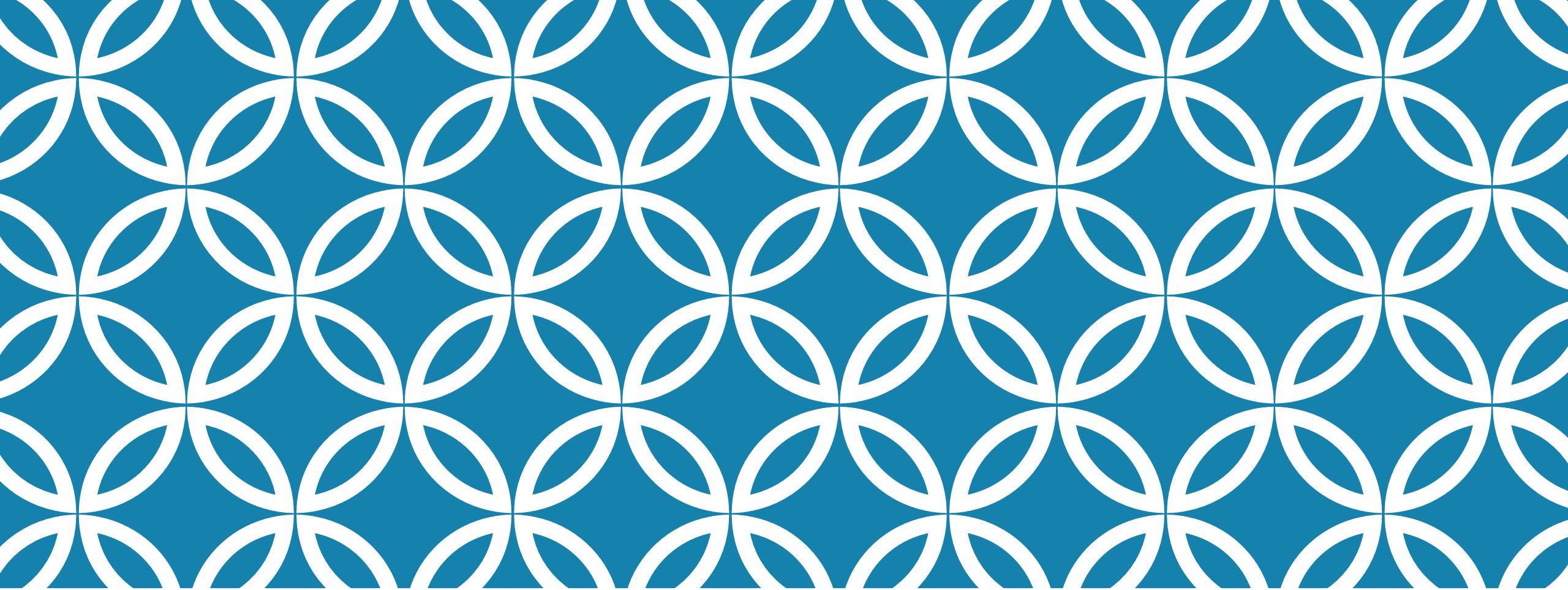


SUPERVISED LEARNING

Dr. Ameni Youssfi Nouira

INTRODUCTION

Supervised learning is a type of machine learning in which a model is trained using labeled data. This means that each training example includes:

- Input data (for example, an image, text, or numbers)
- The correct output (called the label)

The model learns by comparing its predictions with the correct answers and adjusting itself to reduce errors.

➔ Supervised learning is a machine learning method where a model learns from labeled examples to predict the correct output for new data.

INTRODUCTION

Example: If we give a model many emails labeled “spam” or “not spam,” it learns to classify new emails.

If we give house prices with their features (size, location, rooms), it learns to predict the price of a new house.

Main types:

Classification – Predicting categories (e.g., spam or not spam)

Regression – Predicting continuous values (e.g., price, temperature)

Regression Algorithms

REGRESSION

Regression is a machine learning method that models the relationship between variables to predict a continuous output.

In regression, the model learns how input variables (features) are related to a numerical outcome (target). The goal is to find a mathematical function that best fits the data.

Examples:

Predicting house prices based on size, location, and number of rooms.

Predicting temperature for tomorrow.

Predicting stock or cryptocurrency prices.

LINEAR REGRESSION

LINEAR REGRESSION

Linear Regression models the relationship between input features and a continuous target variable by fitting a linear equation.

It assumes a linear relationship between the features and the target.

TYPES OF LINEAR REGRESSION

SIMPLE LINEAR REGRESSION

One independent variable (predictor) predicts the dependent variable (outcome).

Independent Variable (X)



Study Hours

Dependent Variable (Y)



Exam Score

Model:

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

(One predictor)

Example: Study hours → Exam score

More study hours generally leads to higher exam score.

MULTIPLE LINEAR REGRESSION

Two or more independent variables (predictors) predict the dependent variable (outcome).

Independent Variables ($X_1, X_2, \ldots, X_n$)



Study Hours (X_1)



Attendance (X_2)



Sleep Hours (X_3)

Dependent Variable (Y)



Exam Score

Model:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \cdots + \beta_n X_n + \varepsilon$$

(Two or more predictors)

Example: Study hours, attendance, and sleep hours → Exam score

All these factors together influence the exam score.

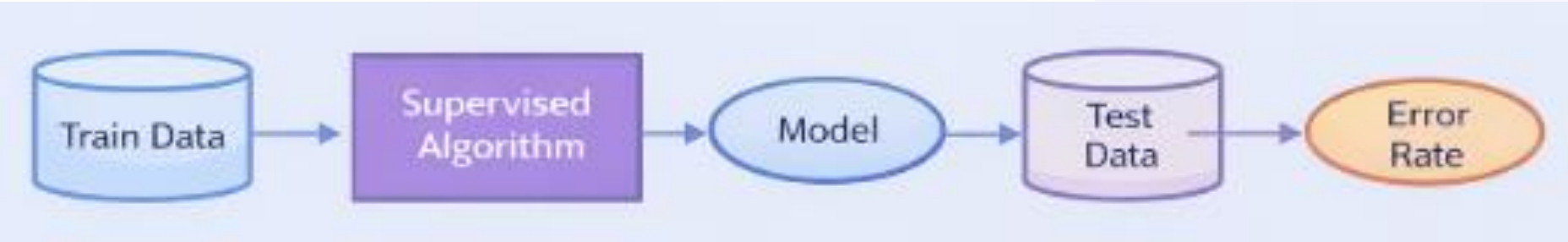
REGRESSION STEPS

Training: from known data with their correct classes (labels).

Test : from unknown (unseen) data

Error rate: number of errors (number of data tested incorrectly by the model) divided by the total of number of test data

$$\text{Error Rate} = \frac{\text{Number of errors}}{\text{Total number of task attempts}}$$



EQUATION OF SAMPLE LINEAR REGRESSION

$Y = wx + b$

Y: represents the dependent variable

X represents the independent variable

w is the slope of the line (how much Y changes for a unit change in X)

LINEAR REGRESSION

The purpose of regression anaLysis is to come up with an equation of a line that fits through that cluster of points with the minimal amount of deviations from the line, the deviation of the points from the line is called « error », Once you have this regression equation, if you knew a person's weight, you could then predict their height, Simple Linear Regression is actually the same as a bivariate correlation between the indepent and dependent variable,

REGRESSION LINE

A regression Line also called a line of best fit, is the line for which the sum of the squares of the residuals is a minimum

The equation of a regression line for an independent variable x and a dependent variable y is:

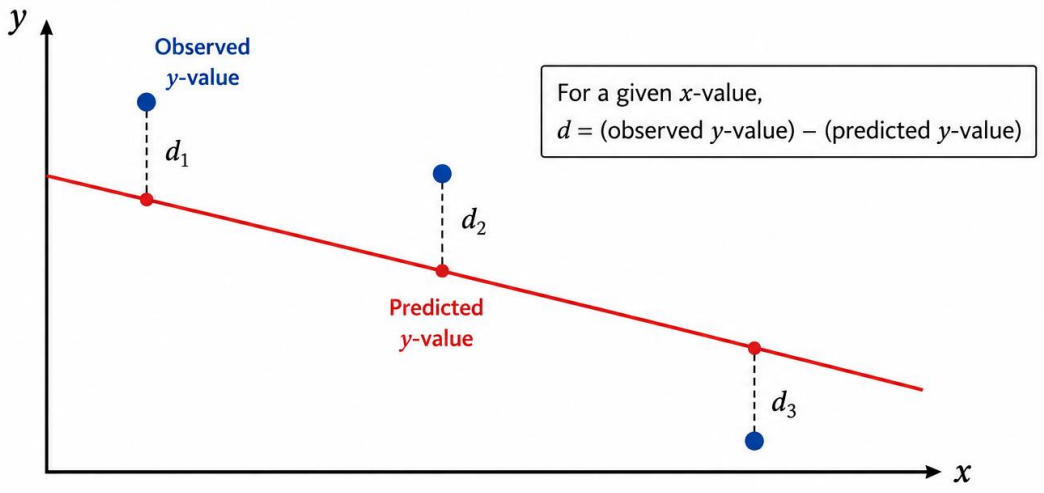
$\hat{y}$= wx+b

Where $\hat{y}$ is the predicted y-value for a given x-value, the slope m and y-intercept b are given by

RISIDUALS

After verifying that the linear correlation between two variables is significant, next we determine the equation of the line that can be used to predict the value of y for a given value of x

Each data point di represents the difference between the ob served y-value and the predicted y-value for a given x-value on the line, these differences are callled residuals



- Observed y-value (actual data point)
- Predicted y-value (value on the regression line)
- --- Residual (difference between observed and predicted)

Residual:
d = observed y-value – predicted y-value
(d can be positive or negative)

COST FUNCTION LINEAR REGRESSION

Our ML model predicts the new value, we aim to have the predicted value as close as possible to the predicted value, if the model predicts the value close to the actual value, then we will say it’s a good model.

So here is where cost function plays an important role, This cost function is needed to calculate the difference between actual and predicted values, So here it is nothing, just the difference between the actual values-predicted values.

REGRESSION LINE

Where $\bar{y}$ is the mean of the y-values and x is the mean of the x-values, the regression line always passes through $(\bar{x}, \bar{y})$

TYPES OF COST FUNCTION IN LINEAR REGRESSION

Mean error: these errors can be negative of positive , therefore, the y can cancel each other out during the summation, and the average error of the model will be zero.

Mean Squared Error: MSE represents the average squared difference between the predictions and expected results

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

Where:

$J(\theta)$= cost function

m= number of training examples

$h_\theta(x^{(i)})$= predicted value

$y^{(i)}$= actual value

θ= model parameters (weights)

USE CASE: PREDICTING HOUSE PRICES BASED ON FEATURES SUCH AS SQUARE FOOTAGE

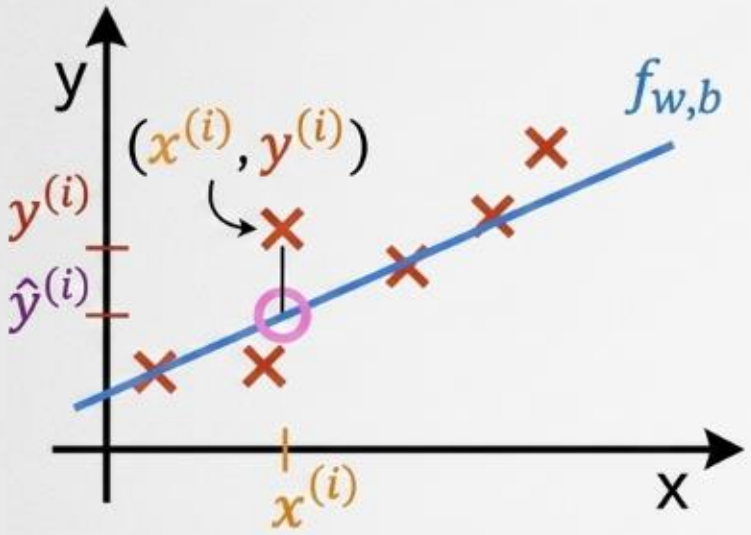
Training set

features size in feet² (x)	targets price \$1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

Model: $f_{w,b}(x) = wx + b$

w, b: parameters

COST FUNCTION WHERE B=0



$$\hat{y}^{(i)} = f_{w,b}(x^{(i)})$$

$$f_{w,b}(x^{(i)}) = wx^{(i)} + b$$

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\Big(\underbrace{\hat{y}^{(i)} - y^{(i)}}_{\text{error}}\Big)^2$$

- m = number of training examples

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)^2$$

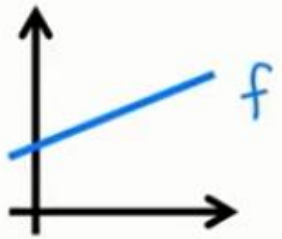
Find w, b:

$\hat{y}^{(i)}$ is close to $y^{(i)}$ for all $(x^{(i)}, y^{(i)})$.

COST FUNCTION WHERE B=0

model:

$$f_{w,b}(x) = wx + b$$



parameters:

w, b

cost function:

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})^2$$

goal:

$$\underset{w,b}{\text{minimize}}\, J(w,b)$$

simplified

$$f_w(x) = wx \qquad b = \emptyset$$

w

$$J(w) = \frac{1}{2m}\sum_{i=1}^{m}(f_w(x^{(i)}) - y^{(i)})^2$$

$wx^{(i)}$

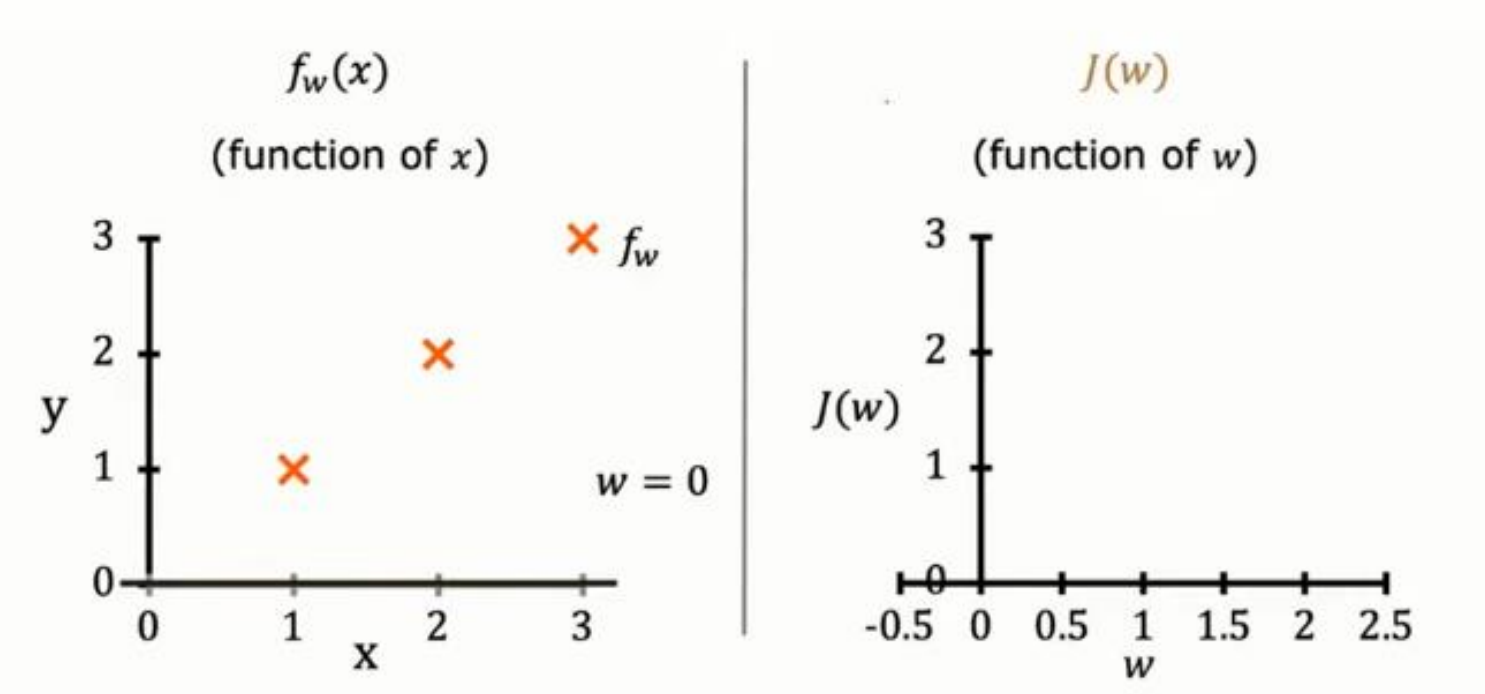
$$\underset{w}{\text{minimize}}\, J(w)$$

COST FUNCTION WHERE B=0

x1(1,1)

x2 (2,2)

x3 (3,3)

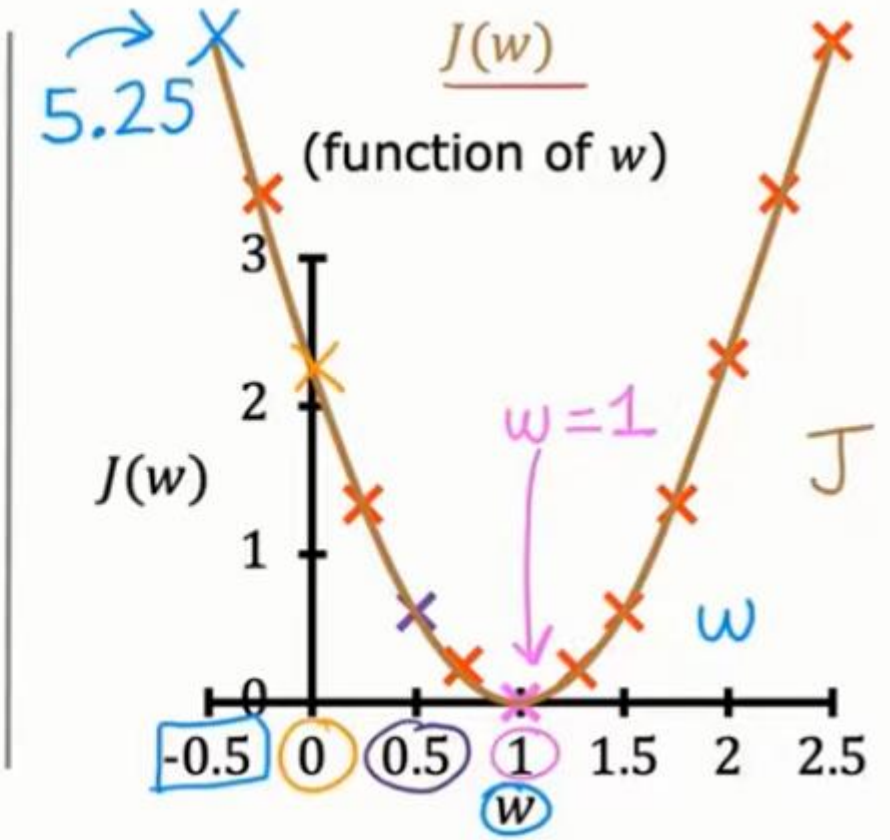


goal of linear regression:

$$\underset{w}{\text{minimize}}\, J(w)$$

general case:

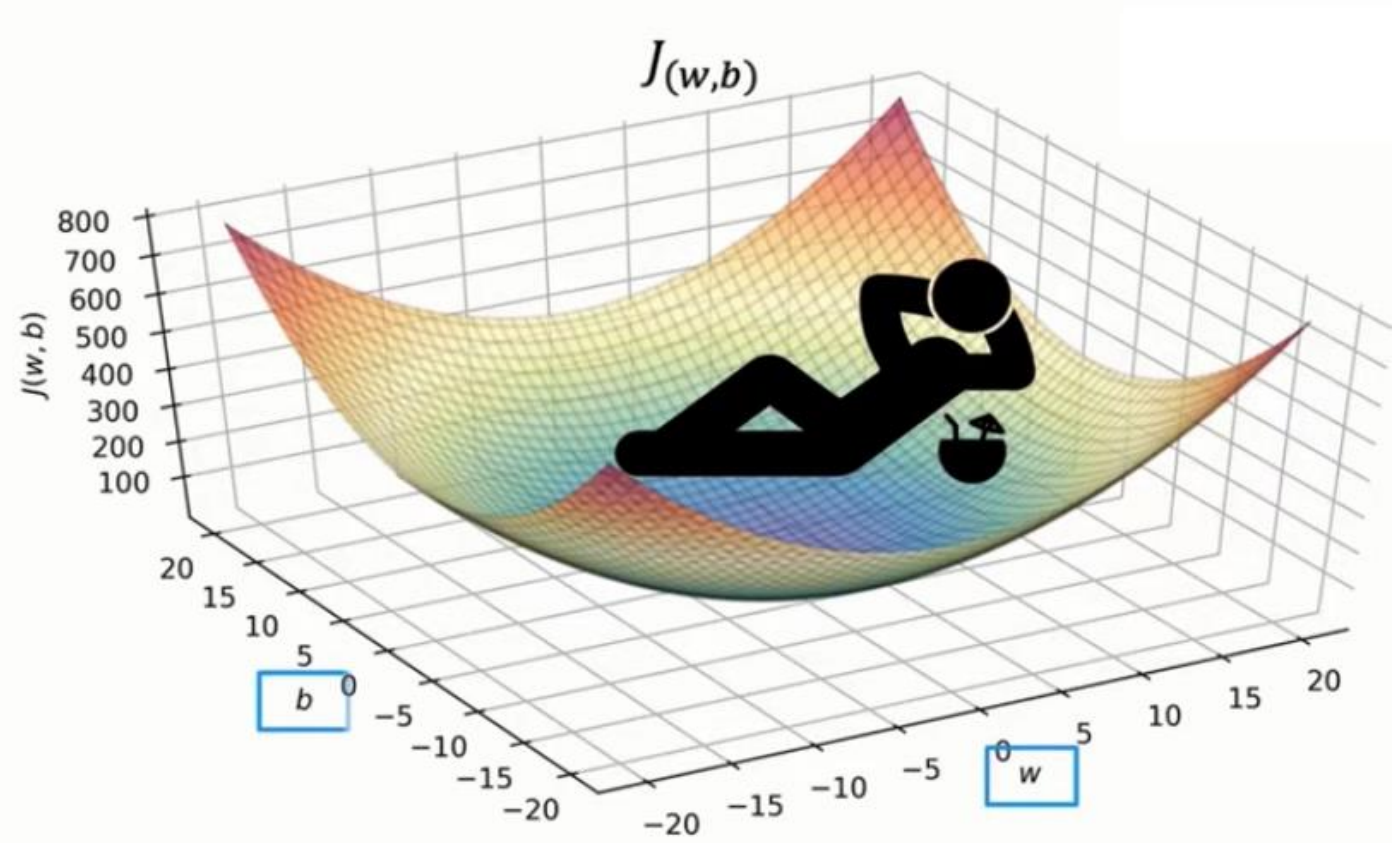
$$\underset{w,b}{\text{minimize}}\, J(w,b)$$

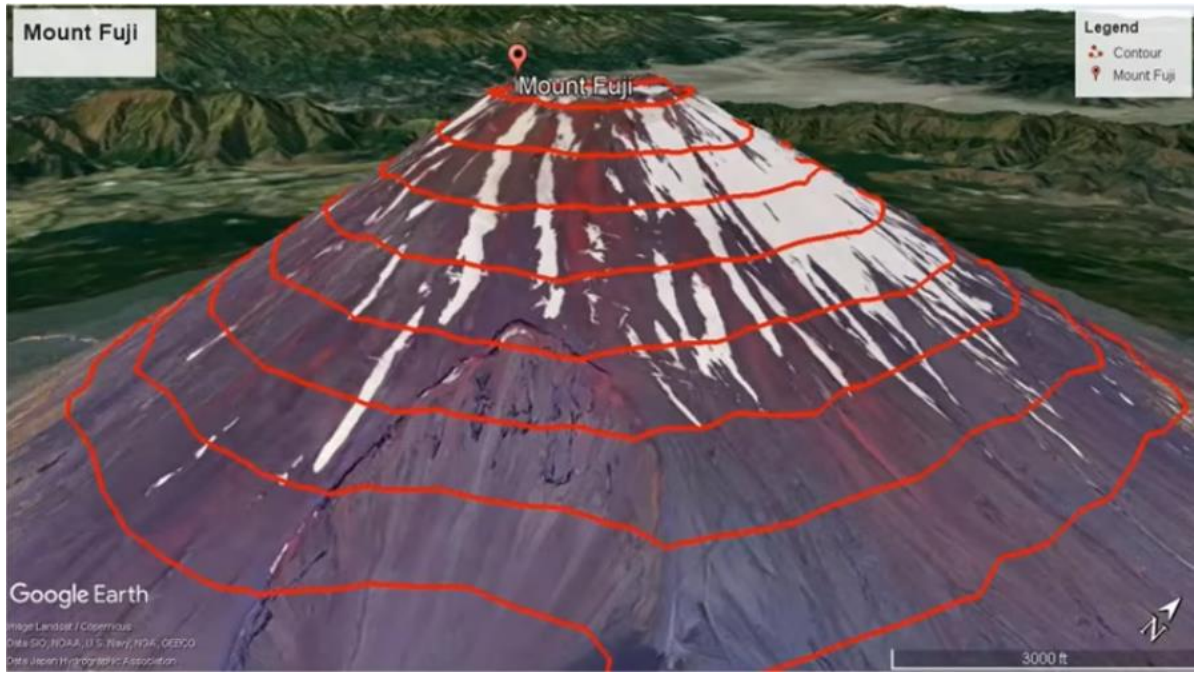


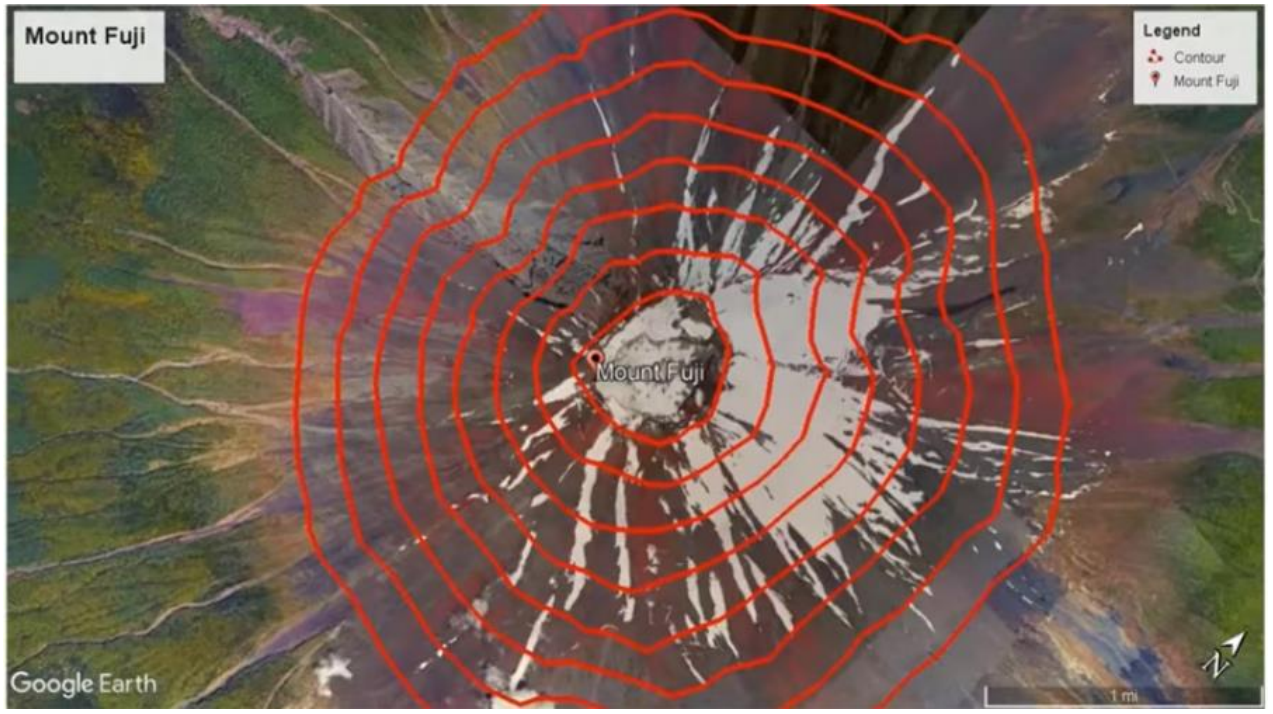
Choose w to minimize J(w)

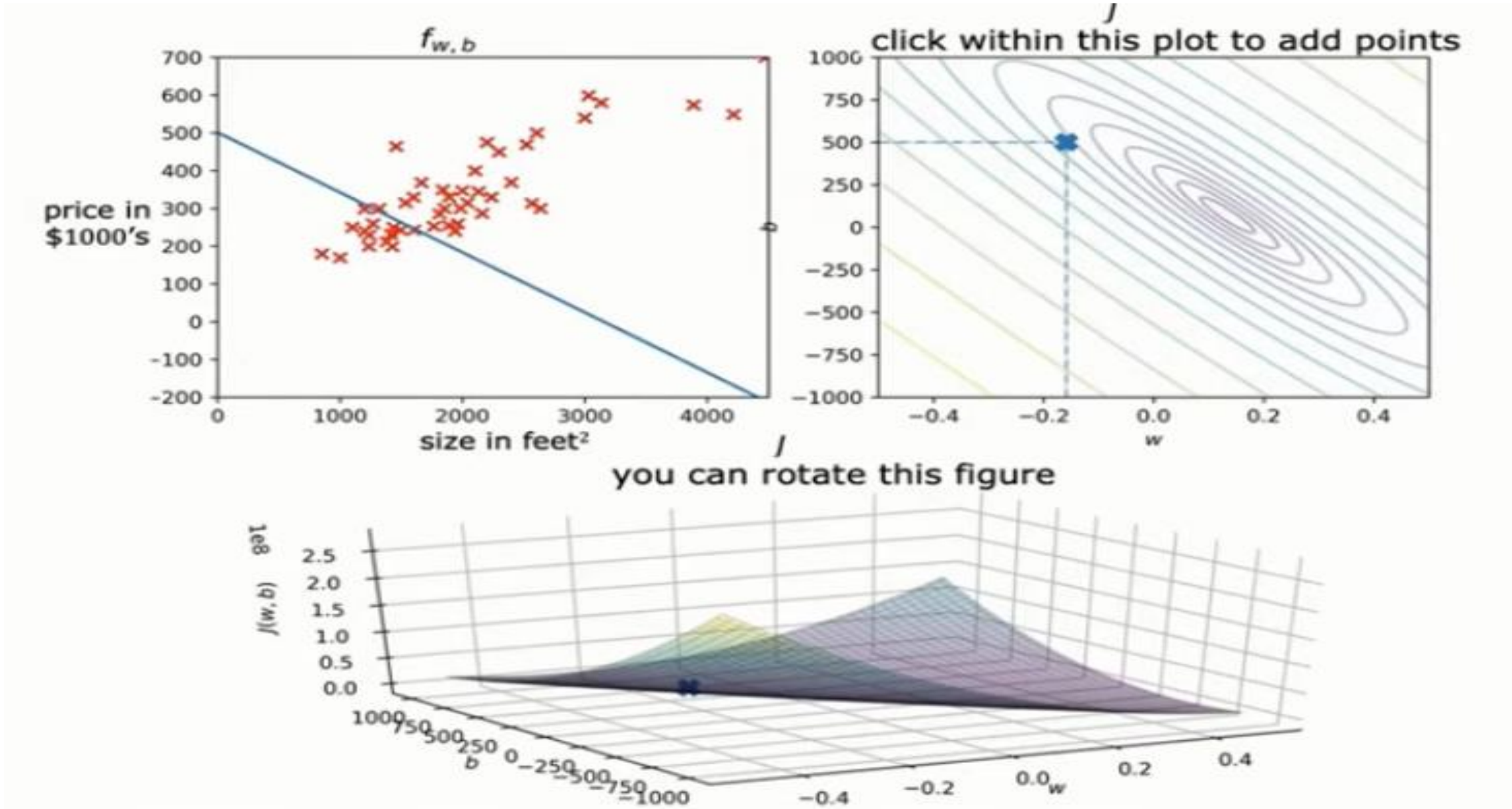
SEE GOOGLE COLAB

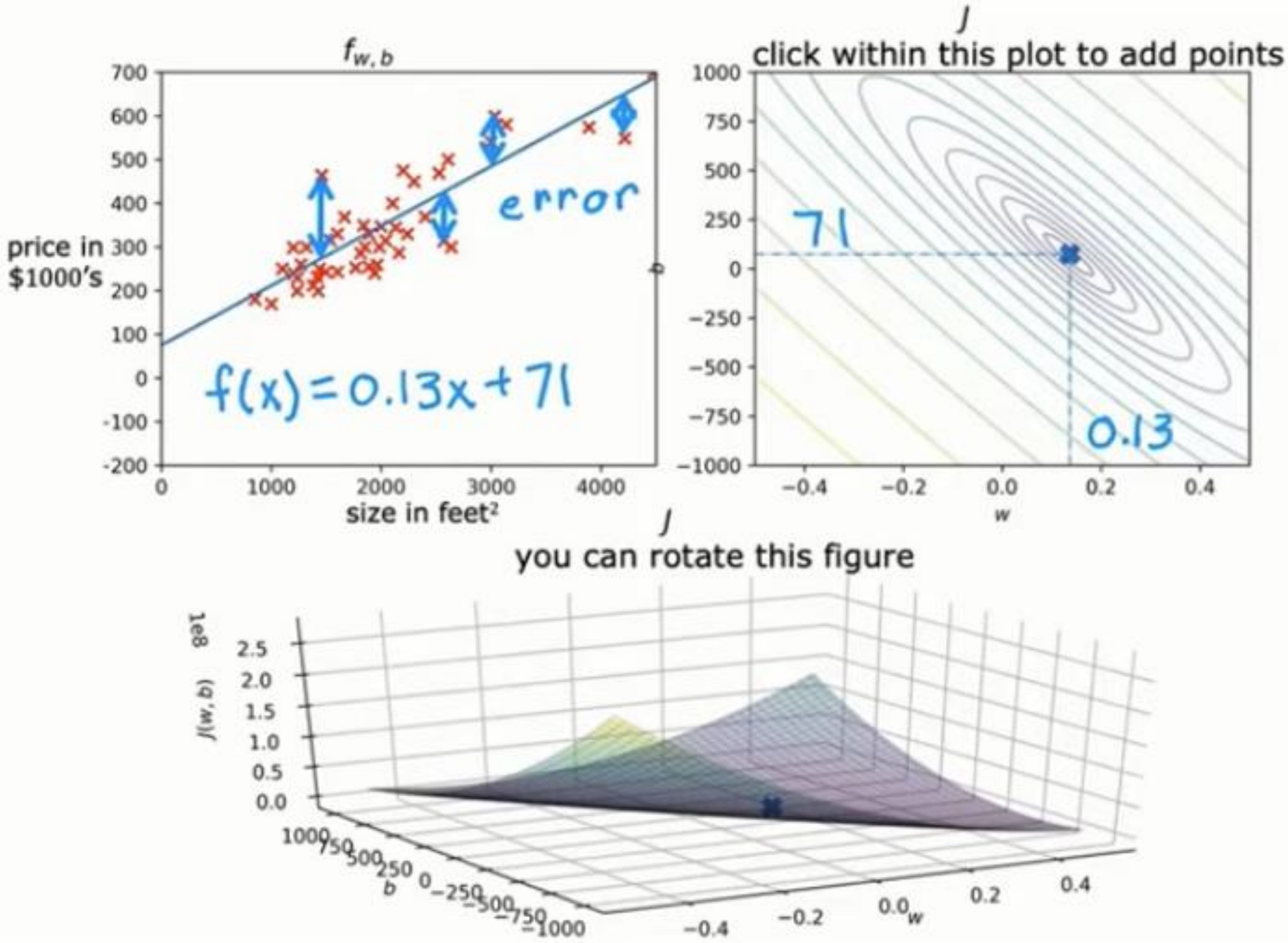
COST FUNCTION $b \neq 0$




Mount Fuji
Legend
Contour
Mount Fuji
Mount Fuji
Google Earth
3000 ft


Mount Fuji
Legend
Contour
Mount Fuji
Mount Fuji
Google Earth
1 mi


$f_{w,b}$
price in $1000's
size in feet²
J
click within this plot to add points
w
b
you can rotate this figure
J(w,b)
1e8


$f_{w,b}$
price in $1000's
size in feet²
error
f(x) = 0.13x + 71
J
click within this plot to add points
71
0.13
w
J
you can rotate this figure
1e8
J(w, b)
b
w

SEE GOOGLE COLAB SLIDE 26

WHY GRADIENT DESCENT?

The cost function only tells us how good or bad the model is, but it does not show how to improve it.

Gradient descent is needed because it provides the direction to update the parameters (w and b) to reduce the error.

Without gradient descent, we cannot efficiently find the minimum value of the cost function in complex models.

Therefore, the cost function measures performance, while gradient descent is the optimization method that actually improves the model.

GRADIENT DESCENT OVERVIEW

Have some function J(w,b)

Want minJ(w,b)

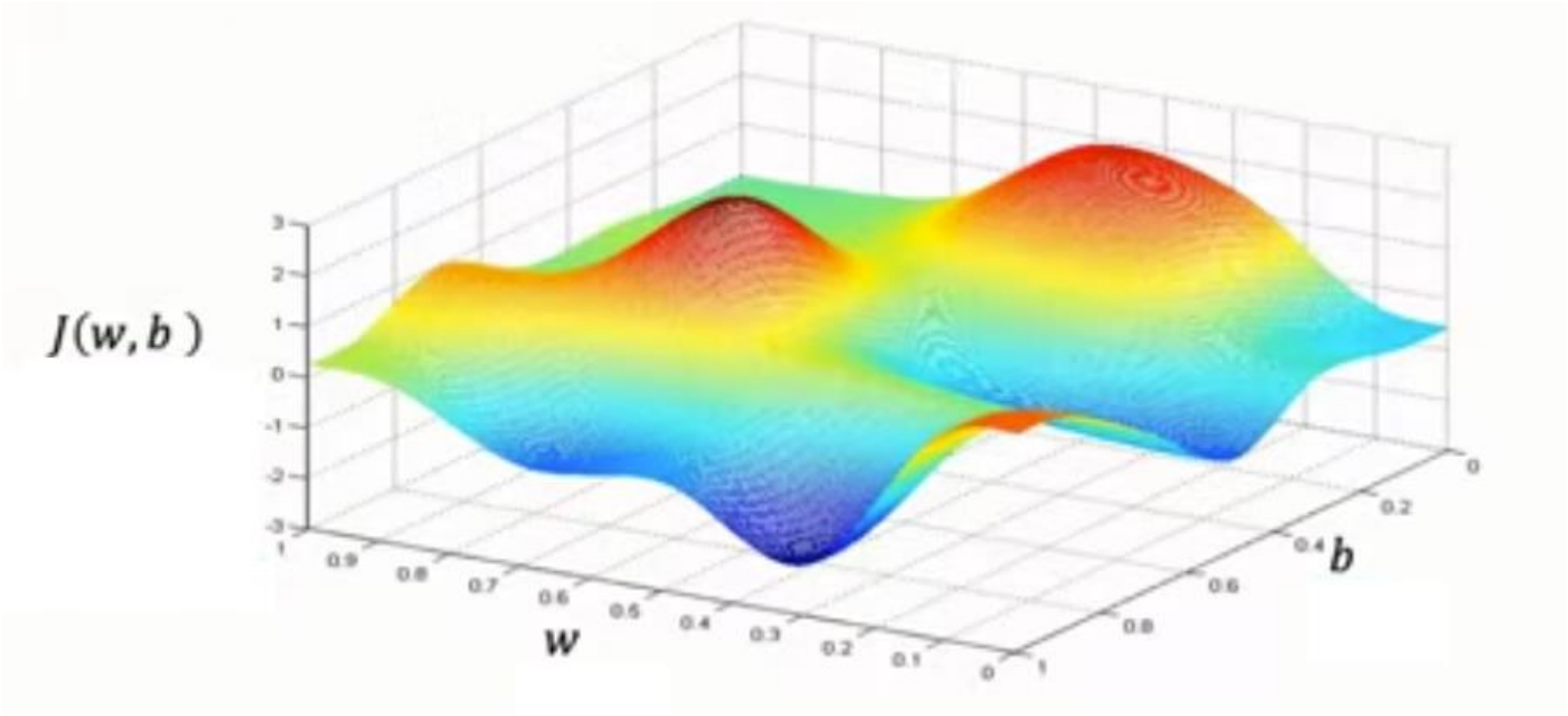
Outline:

Start with some w,b

Keep changing w,b to reduce J(w,b)

Until we settle at or near a minimum

CONTD...



GRADIENT DESCENT ALGORITHM

- Repeat until convergence:

$$w = w - \alpha \frac{\partial}{\partial w} J(w,b)$$
$$b = b - \alpha \frac{\partial}{\partial b} J(w,b)$$

- Labels:

Learning rate (α)

Derivative ($\frac{\partial}{\partial w} J(w,b)$)

Simultaneously update w and b

- Correct: Simultaneous update

$$tmp_w = w - \alpha \frac{\partial}{\partial w} J(w,b)$$
$$tmp_b = b - \alpha \frac{\partial}{\partial b} J(w,b)$$
$$w = tmp_w$$
$$b = tmp_b$$

Incorrect

$$tmp_w = w - \alpha \frac{\partial}{\partial w} J(w,b)$$
$$w = tmp_w$$
$$tmp_b = b - \alpha \frac{\partial}{\partial b} J(w,b)$$
$$b = tmp_b$$

CONTD...

$$\frac{\partial}{\partial w} J(w,b) = \frac{\partial}{\partial w} \frac{1}{2m} \sum_{i=1}^{m} \left(f_{w,b}\left(x^{(i)}\right) - y^{(i)} \right)^2 = \frac{\partial}{\partial w} \frac{1}{2m} \sum_{i=1}^{m} \left(wx^{(i)} + b - y^{(i)} \right)^2$$

$$= \frac{1}{2m} \sum_{i=1}^{m} 2 \left(wx^{(i)} + b - y^{(i)} \right) x^{(i)}$$

$$= \frac{1}{m} \sum_{i=1}^{m} \left(f_{w,b}\left(x^{(i)}\right) - y^{(i)} \right) x^{(i)}$$

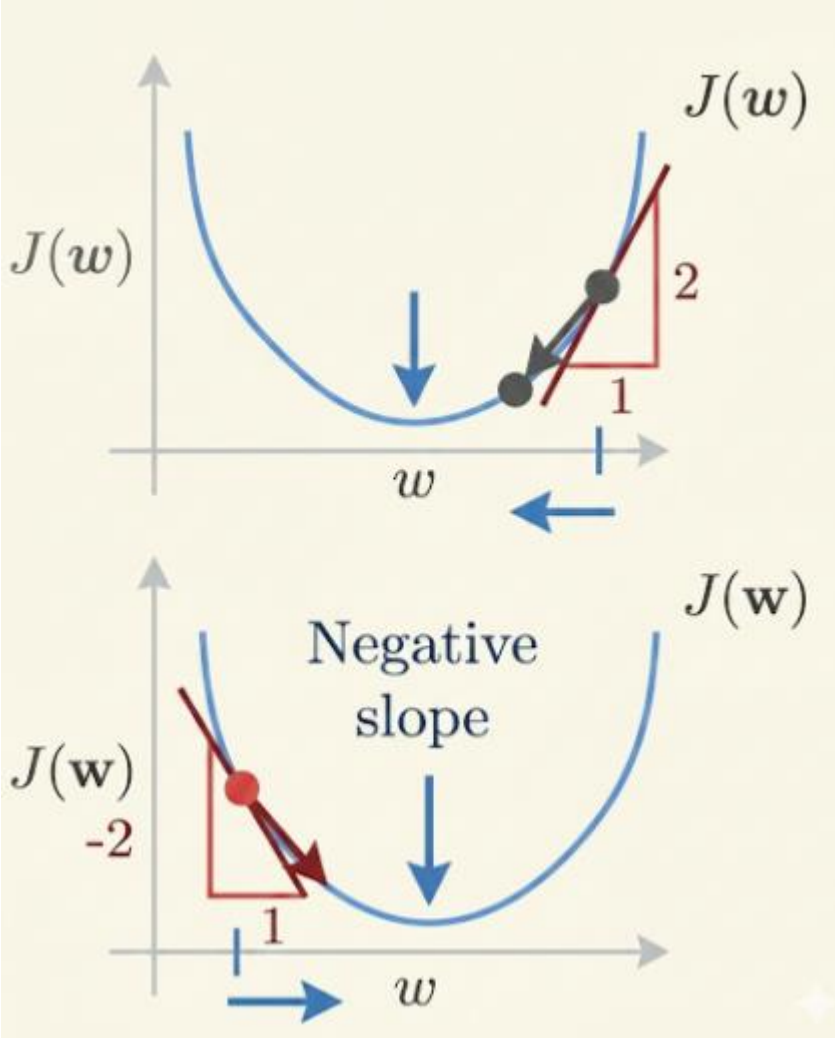
CONTD...

$$\frac{\partial}{\partial b} J(w,b) = \frac{\partial}{\partial b} \left[\frac{1}{2m} \sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)})^2 \right]$$

$$= \frac{\partial}{\partial b} \left[\frac{1}{2m} \sum_{i=1}^{m} (wx^{(i)} + b - y^{(i)})^2 \right]$$

$$= \frac{1}{2m} \sum_{i=1}^{m} 2(wx^{(i)} + b - y^{(i)}) \cdot 1$$

$$= \frac{1}{m} \sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)})$$



Gradient Descent Update Rule

$w = w - \alpha \, \partial J(w)/\partial w$

Case 1: $\partial J(w)/\partial w > 0$

$w = w - \alpha \times$ (positive number)

$\rightarrow$ w decreases

Case 2: $\partial J(w)/\partial w < 0$

$w = w - \alpha \times$ (negative number)

$\rightarrow$ w increases

EXAMPLE W=0,B=0, ALPHA=0,1

$w := w - \alpha \cdot \partial J/\partial w$
$b := b - \alpha \cdot \partial J/\partial b$

α is the learning rate

$\partial J/\partial w = {}^{1}/_{m} \cdot \Sigma (\hat{y}_i - y_i) \cdot x_i$
$\partial J/\partial b = {}^{1}/_{m} \cdot \Sigma (\hat{y}_i - y_i)$

(x,y) =(1,2)

(x,y) =(2,4)

(x,y) =(3,5)

(x,y) =(4,8)

$\bar{y}$=wx+b

$\bar{y}_1$-y=0-2=-2

$\bar{y}$2-y=0-4=-4

$\bar{y}$3-y=0-5=-5

$\bar{y}$4-y=0-8=-8

$\partial J/\partial w = {}^{1}/_{m} \cdot \Sigma (\hat{y}_i - y_i) \cdot x_i$

=1/4[(-2*1)+(-4*2)+(-5*3)+(-8*4)]

=1/4(-57)=-14,25

w_new=0-0,1*(-14,25)=+1,425

b=0; $\partial J/\partial b = {}^{1}/_{m} \cdot \Sigma (\hat{y}_i - y_i)$

b=1/4(-19)=-4,75

b_new=0-0,1*(-4,75)= +0,475

$J(w,b) = 1/2m \Sigma (\hat{y}_i - y_i)^2$

$J(w,b)=J(0,0)=1/8[(0-2)^2 +(0-4)^2+(0-5)^2+(0-8)^2]=13,625$

Updated parameters after one Gradient Descent step:

w = 1.425
b = 0.475

NEXT STEP COMPUTE THE PREDICTIONS AGAIN USING THE UPDATED PARAMETERS

Prediction formula:
$\hat{y} = wx + b$
$\hat{y} = 1.425x + 0.475$

Predictions:

$x=1 \rightarrow \hat{y}=1{,}425*1 + 0{,}475=1.9$
$x=2 \rightarrow \hat{y}=3.325$
$x=3 \rightarrow \hat{y}=4.75$
$x=4 \rightarrow \hat{y}=6.175$

Errors:

$\hat{y}1-y1 = 1.9-2 = -0.1$
$\hat{y}2-y2 = 3.325-4 = -0.675$
$\hat{y}3-y3 = 4.75-5 = -0.25$
$\hat{y}4-y4 = 6.175-8 = -1.825$

New Cost Function:
$J(w,b) = 1/(2m)\ \Sigma(\hat{y}i - yi)^2$
$J(1.425,0.475)$
$= 1/8\ [(-0.1)^2 + (-0.675)^2 + (-0.25)^2 + (-1.825)^2]$
$= 1/8\ [0.01 + 0.455625 + 0.0625 + 3.330625]$
$= 1/8\ (3.85875)$
$= 0.4823$

RESULT:
Initial Cost : $J(0,0) = 13.625$
Updated Cost : $J(1.425,0.475) = 0.4823$

CONT...

Iteration 0 : J = 13.625

Iteration 1 : J = 0.4823

Iteration 2 : J =

...

Minimum Cost → Best Fit Line

Since the cost decreased from 13.625 to 0.4823,
the model moved closer to the optimal values of w and b.
Repeat the process until J(w,b) reaches its minimum value.

HOW TO CHOOSE LEARNING RATE?

LEARNING RATE (α)

$w = w - \alpha \times \partial J(w)/\partial w$

α too small:

✓ Slow convergence

✓ Many iterations required

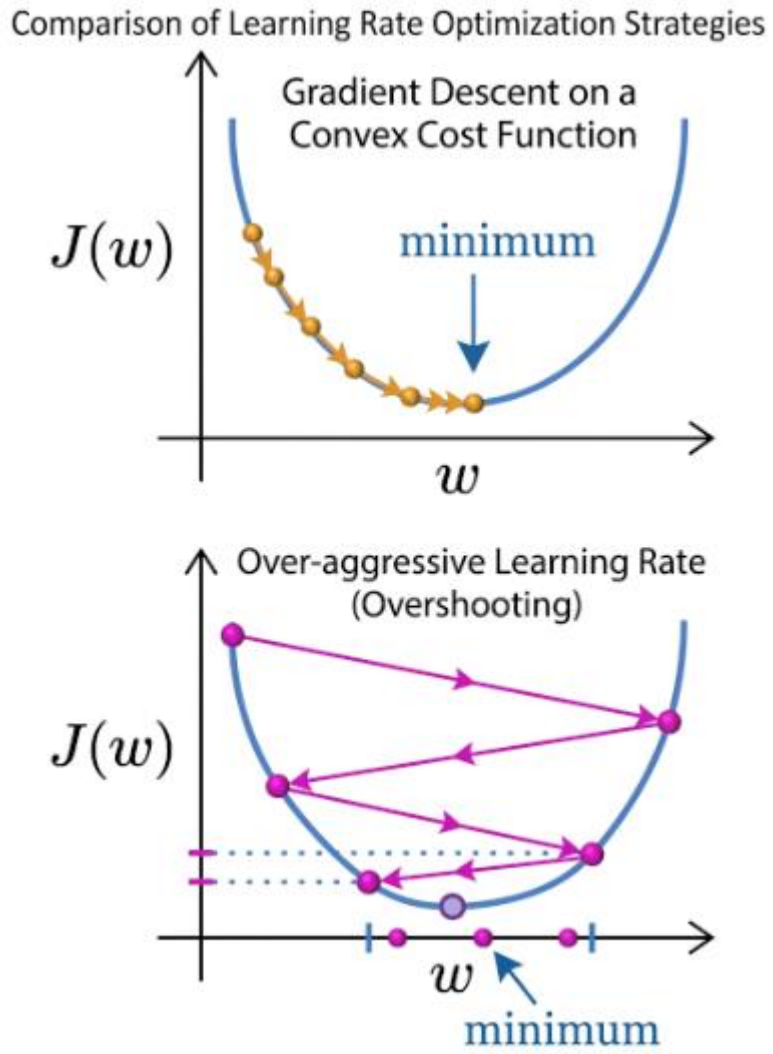
α too large:

✗ Overshoots the minimum

✗ May oscillate

✗ May diverge

A good learning rate leads to fast and stable convergence.



TYPES OF COST FUNCTION IN LINEAR REGRESSION

Mean Absolute Error: MAE is a variation of MAE that square difference instead of taking the absolute value of the difference. There is no possibility of negative errors.

I= index of sample

^y = predicted value

Y= expected value

M= number of samples in the dataset

$$MAE = \frac{1}{m}\sum_{i=1}^{m}|\hat{y}^{(i)} - y^{(i)}|$$

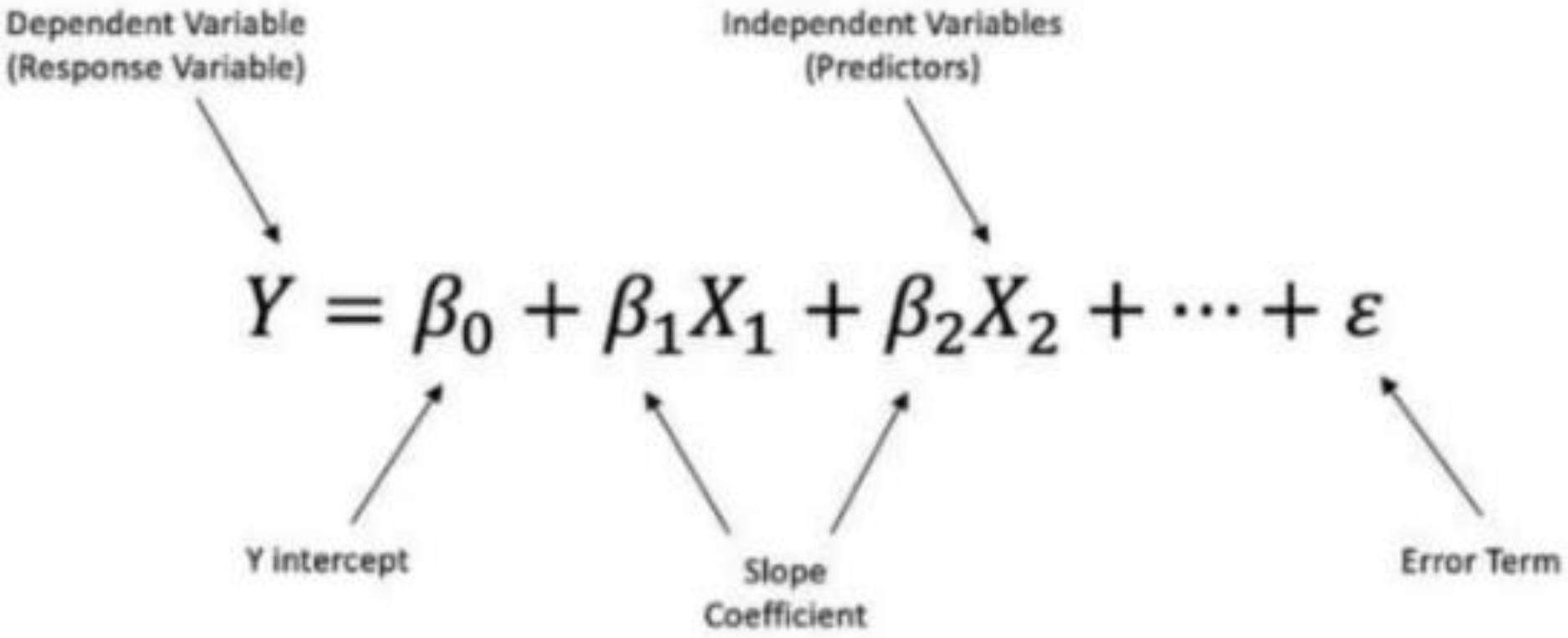
TYPES OF COST FUNCTION IN LINEAR REGRESSION

Root Mean Squared Error (RMSE): the Root Mean Squared Error (RMSE) is one of the two main performance indicators for a regression model, it measures the average difference, between values predicted by a model and the actual values

$$RMSE = \sqrt{\frac{\sum_{i=1}^{N} (Predicted_i - Actual_i)^2}{N}}$$

LAB datamining

MULTIPLE LINEAR REGRESSION



MULTIPLE LINEAR REGRESSION

Multiple features (variables)

Size in feet^2	Number of bedrooms	Number of floors	Age of home in years	Price ($) in $1000's
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

MULTIPLE LR

Model:

Previously: $f_{w,b}(x) = wx + b$

$$f_{w,b}(x) = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$$

$$f_{w,b}(x) = 0.1x_1 + 4x_2 + 10x_3 - 2x_4 + 80$$

x_1: size; x_2: #bedrooms; x_3: #floors; x_4: years; 80: base price

$$f_{w,b}(x) = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

MULTIPLE LR

$$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

$$\vec{w} = \begin{bmatrix} w_1 & w_2 & w_3 & \cdots & w_n \end{bmatrix}$$ parameters of the model

b is a number

vector $\vec{x} = \begin{bmatrix} x_1 & x_2 & x_3 & \cdots & x_n \end{bmatrix}$ vector

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b = w_1x_1 + w_2x_2 + w_3x_3 + \cdots + w_nx_n + b$$

MULTIPLE LR

1.Gradient Descent

We update parameters step by step:

$$\beta_j := \beta_j - \alpha \frac{\partial J}{\partial \beta_j}$$

Repeated until the cost function is minimized.

✓ Works for big datasets
✓ Used in machine learning models

MULTIPLE LR

2. Analytical solution

$$\beta = (X^T X)^{-1} X^T y$$

This directly gives optimal values of β

✓ Exact solution
✗ Slow for very large datasets

EXAMPLE OF MULTIPLE LINEAR REGRESSION

Here the matrices for Y and X are givenas follows:

$$X = \begin{Bmatrix} 114 \\ 125 \\ 138 \\ 142 \end{Bmatrix} \qquad Y = \begin{Bmatrix} 1 \\ 6 \\ 8 \\ 12 \end{Bmatrix}$$

Weekly sales depend on both Product 1 and Product 2 sales.

$Y=\beta 0+\beta 1X1+\beta 2X2$

EXAMPLE OF MULTIPLE LINEAR REGRESSION

How do we find the coefficients?

Using the normal equation:

$$\beta = (X^T X)^{-1} X^T Y$$

This formula gives the values of:

$$\beta_0, \beta_1, \beta_2$$

That minimize the squared error.

$$X^T X = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \\ 4 & 5 & 8 & 2 \end{pmatrix} \begin{pmatrix} 1 & 1 & 4 \\ 1 & 2 & 5 \\ 1 & 3 & 8 \\ 1 & 4 & 2 \end{pmatrix} = \begin{pmatrix} 4 & 10 & 19 \\ 10 & 30 & 46 \\ 19 & 46 & 109 \end{pmatrix}$$

$$(X^T X)^{-1} = \begin{pmatrix} 4 & 10 & 19 \\ 10 & 30 & 46 \\ 19 & 46 & 109 \end{pmatrix}^{-1} = \begin{pmatrix} 3.15 & -0.59 & -0.30 \\ -0.59 & 0.20 & 0.016 \\ -0.30 & 0.016 & 0.054 \end{pmatrix}$$

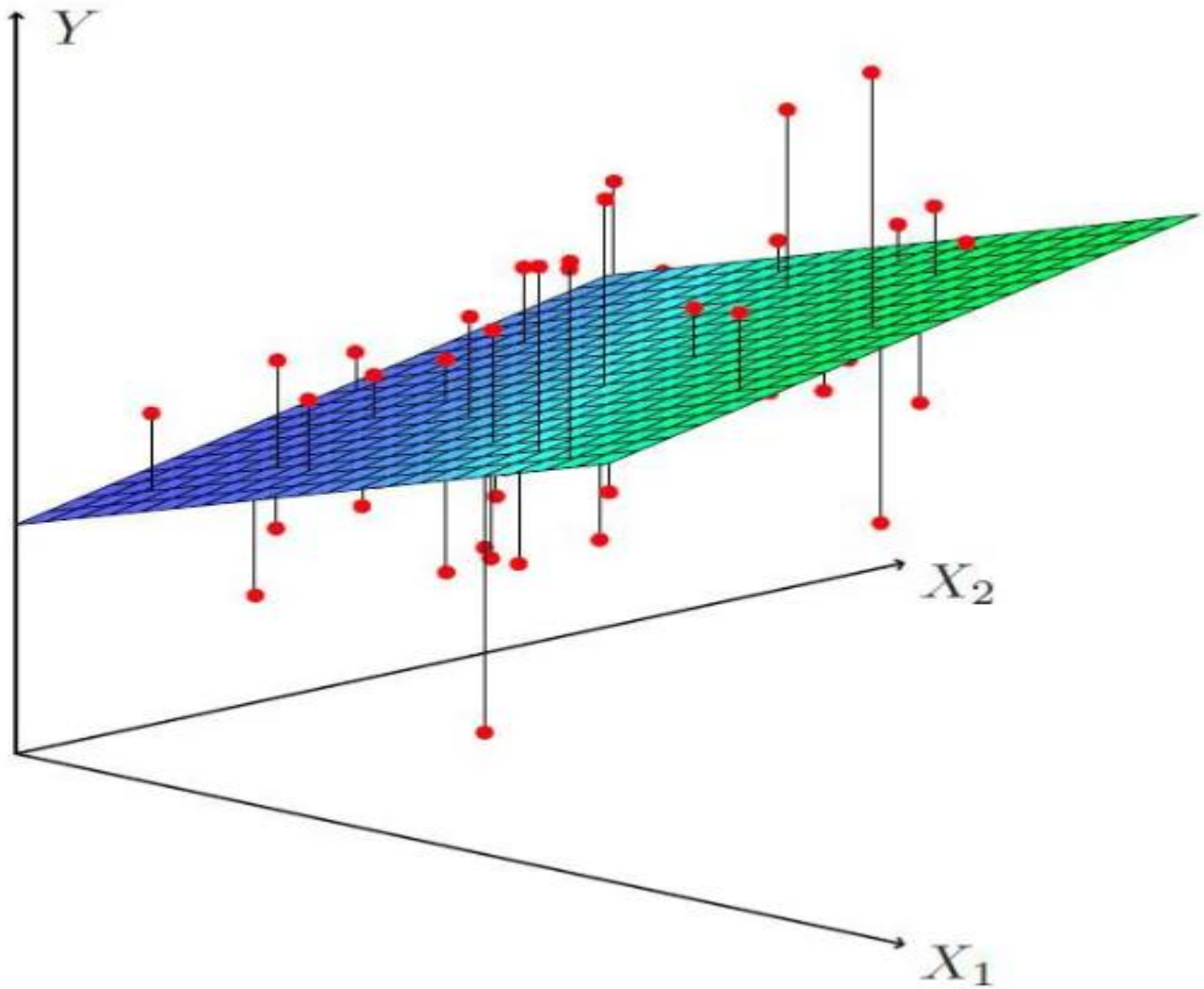
$$X^T X)^{-1} X^T = \begin{pmatrix} 3.15 & -0.59 & -0.30 \\ -0.59 & 0.20 & 0.016 \\ -0.30 & 0.016 & 0.054 \end{pmatrix} \times \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \\ 4 & 5 & 8 & 2 \end{pmatrix} = \begin{pmatrix} 0.05 & 0.47 & -1.02 & 0.19 \\ -0.32 & -0.098 & 0.155 & 0.26 \\ -0.065 & 0.005 & 0.185 & -0.125 \end{pmatrix}$$

EXAMPLE OF MULTIPLE LINEAR REGRESSION

$$\beta = ((X^T X)^{-1} X^T) Y = \begin{pmatrix} 0.05 & 0.47 & -1.02 & 0.19 \\ -0.32 & -0.098 & 0.155 & 0.26 \\ -0.065 & 0.005 & 0.185 & -0.125 \end{pmatrix} \times \begin{pmatrix} 1 \\ 6 \\ 8 \\ 12 \end{pmatrix} = \begin{pmatrix} -1.69 \\ 3.48 \\ -0.05 \end{pmatrix}$$

$$\beta_0 = -1,69$$
$$\beta_1 = 3,48$$
$$\beta_2 = -0,05$$

Y= -1,69 + 3,48x1+(-0,05)x4


Y
X2
X1

Lab Multiple Linear regression

POLYNOMIAL REGRESSION

POLYNOMIAL REGRESSION

- Polynomial Regression is a form of linear regression where the relationship between the independent variable (x) and the dependent variable (y) is modelled as an nth degreee polynomial
- It is useful when the data exhibits a non-lineat relationship allowing the model to fit a curve to the data
- A simple polynomial regression algorithm only works when the relationship between the data is linear, But suppose we have non-linear data, then linear regression will not be able to draw a best fit line

POLYNOMIAL REGRESSION

Polynomial Regression does not require the relationship between the independent and dependent variables to be linear in the data set.

This is also one of the main difference between the Linear and Polynomial Regression.

Polynomial Regression is generally used when the points in the data are not captured by the Linear Regression Model and the LR fails in describing the best result clearly.

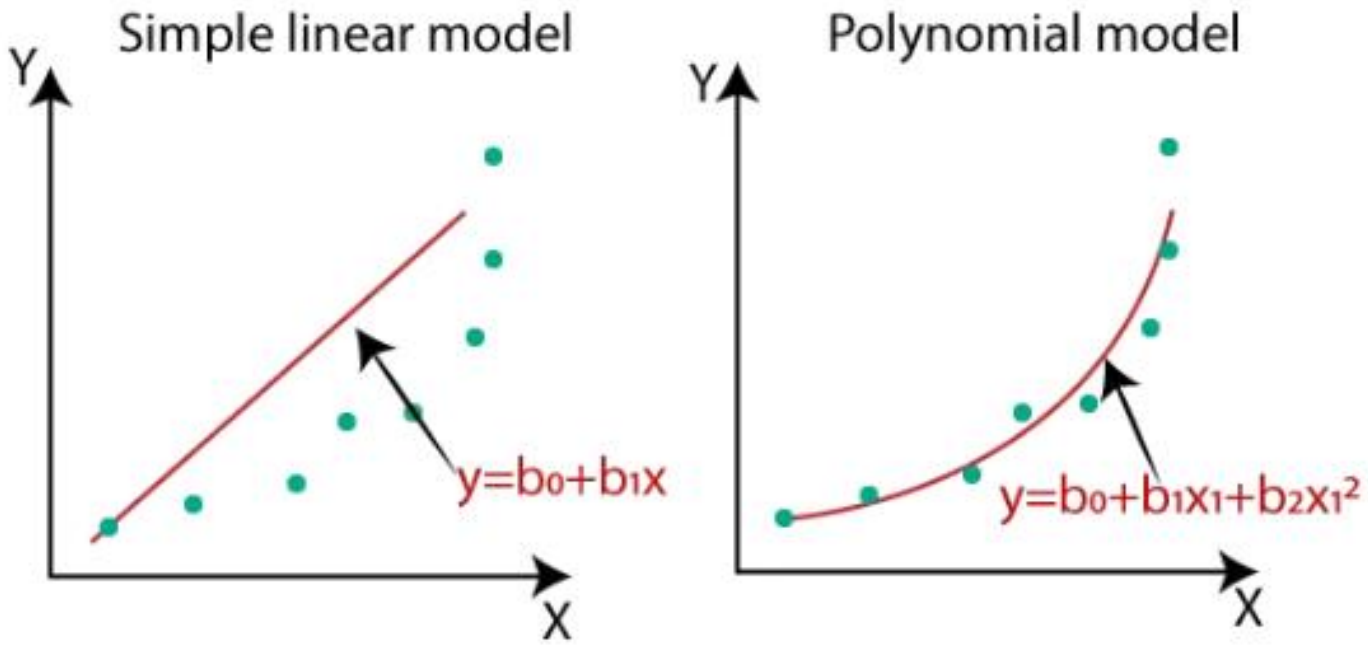
For example:

Predicting the growth of a plant over time.

Analyzing temperature changes throughout the year.

Modeling stock prices that follow cyclical patterns.

LINEAR REGRESSION VS PLYNOMIAL REGRESSION



As shown in the figure a straight line will not properly fit this kind of data. So, how can we find a model that represents the data more accurately?

POLYNOMIAL REGRESSION

Definition: This regression models a non-linear relationship between a single independent variable and the dependent variable by raising the variable to powers.

General form (for 1 variable x):

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \ldots + \beta_d x^d + \varepsilon$$

Example: Predicting plant growth over time, where growth is not linear.

Even though the model is non-linear in x, it is still linear in the coefficients (β), so standard linear regression techniques can be used to estimate it.

POLYNOMIAL REGRESSION

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \ldots + \beta_d x^d + \varepsilon$$

Where:

y= dependent variable (target)

x= independent variable (feature)

d= degree of the polynomial (1 = linear, 2 = quadratic, 3 = cubic, etc.)

$\beta_0, \beta_1, \ldots, \beta_d$ =coefficients to be learned

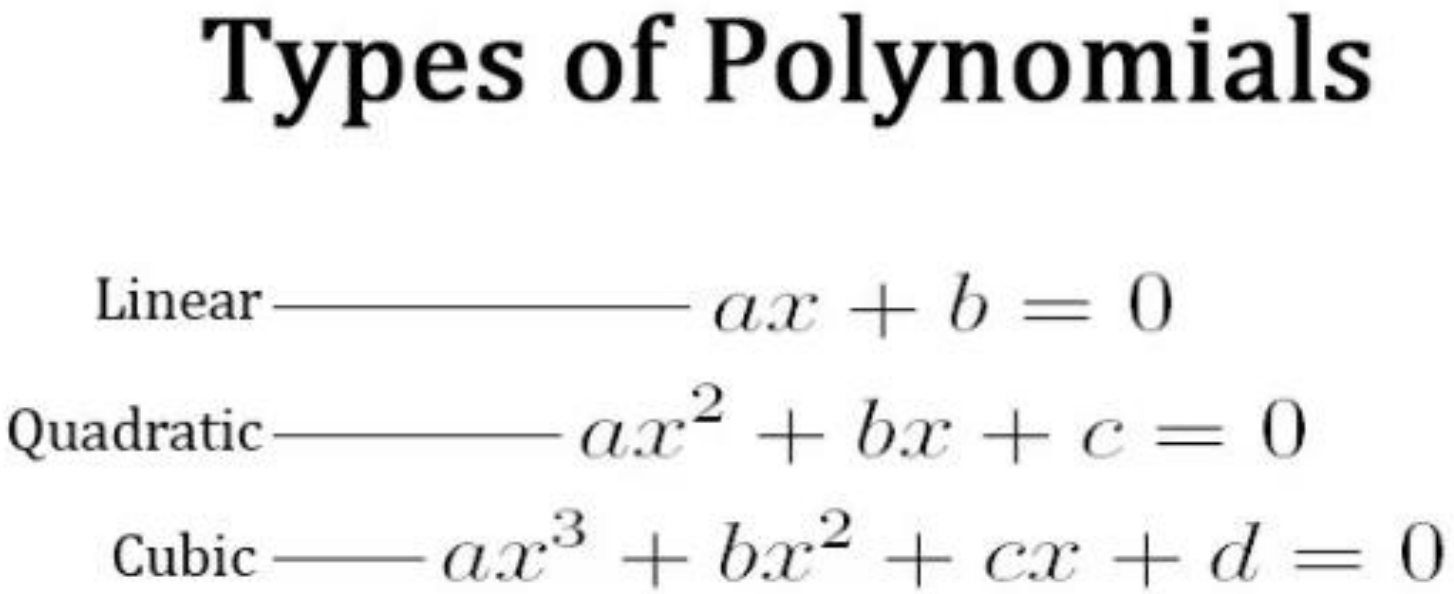
ε= error term

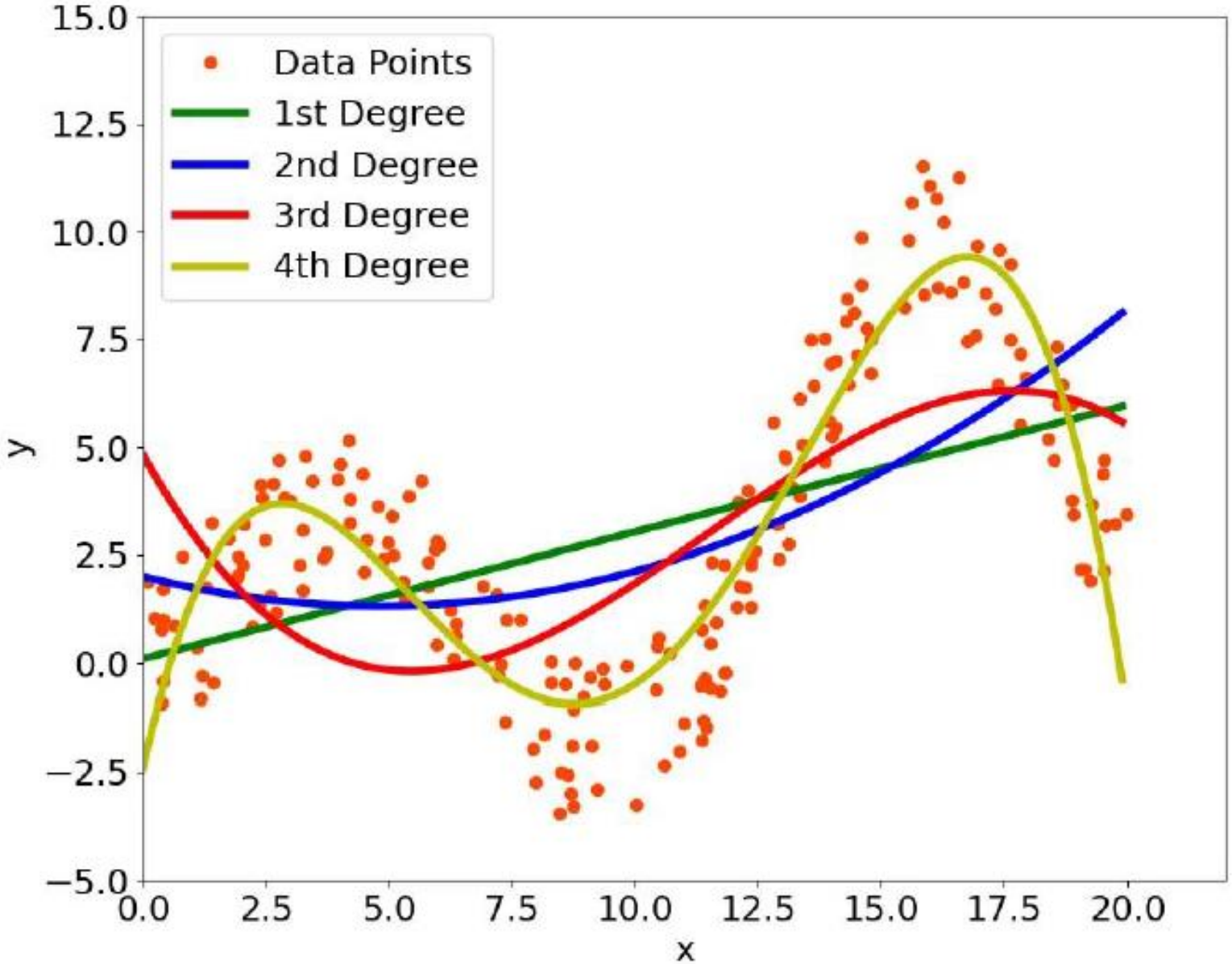
DEGREE OF POLYNOMIAL

The degree n defines the complexity of the curve. For example:

- A **degree 1** polynomial (i.e., $y=\beta 0+\beta 1x$) is just a linear regression model.
- A **degree 2** polynomial is quadratic ($y=\beta 0+\beta 1x+\beta 2x2$) and can capture parabolic relationships.
- A **degree 3** or higher will model even more complex relationships, introducing cubic terms, quartic terms, etc.

POLYNOMIAL REGRESSION TYPES




Data Points
1st Degree
2nd Degree
3rd Degree
4th Degree
15.0
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
y
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
x

WHEN USE POLYNOMIAL REGRESSION

- Use **polynomial regression** in situations where the relationship between your independent variable(s) and the dependent variable is **non-linear,** meaning it cannot be well represented by a straight line, but can be approximated by a curve.

1. Curved trends in data:
   - ✓Growth processes (e.g., plant growth over time).
   - ✓Population growth or decay that accelerates or slows down.
2. Seasonal or cyclical patterns:Temperature changes over the year. Sales or stock prices that follow repeating cycles.
3. Data with turning points:Situations where the dependent variable increases, then decreases (or vice versa) within the range of your data.

Lab Polynomial Regression